

New patterns operators for CEQL

Cristian Riveros ✉

Pontificia Universidad Católica de Chile

Abstract

In this draft, we formalize some new patterns operators for CEQL to be implemented in CORE.

2012 ACM Subject Classification Theory of computation → Database theory

Keywords and phrases Streams, complex event recognition, CEQL, CORE

1 Preliminaries

Sets, intervals, and mappings. Given a set A , we denote by $\mathcal{P}(A)$ the set of all finite subsets of A . We denote by $\mathbb{N}$ the natural numbers. Given $n, m \in \mathbb{N}$ with $n \leq m$, we denote by $[n]$ the set $\{1, \dots, n\}$ and by $[n..m]$ the interval $\{n, n+1, \dots, m\}$ over $\mathbb{N}$. As usual, we write $f : A \rightarrow B$ to denote a *function* f from the set A to B where every element in A has an image. A *mapping* M is a partial function that maps a finite number of elements from A to elements over B . We write $M : A \mapsto B$ to denote a mapping M from A to B . We denote by $\text{dom}(M)$ the domain of M (i.e., all $a \in A$ such that $M(a)$ is defined), and by $\text{img}(M)$ the image of M . We will usually use the notation $[a_1 \mapsto b_1, \dots, a_k \mapsto b_k]$ to define a mapping M with $\text{dom}(M) = \{a_1, \dots, a_k\}$, $\text{img}(M) = \{b_1, \dots, b_k\}$ and $M(a_i) = b_i$ for every $i \in [k]$. Furthermore, for a map M and $a \notin \text{dom}(M)$ we write $[M, a \mapsto b]$ to specify a new map M' that extends M mapping a to b .

Bags. A *bag* or *multiset* (with own identity) B is a mapping $B : I \mapsto U$ where $I = \text{dom}(B)$ is a finite set of identifiers (or ids) and $U = \text{img}(B)$ is the underlying set of the bag. Given any bag B , we refer to these components as $I(B)$ and $U(B)$, respectively. For example, a bag $B = \{\{a, a, b\}\}$ (where a is repeated twice) can be represented with a mapping $B_0 = [1 \mapsto a, 2 \mapsto a, 3 \mapsto b]$ where $I(B_0) = \{1, 2, 3\}$ and $U(B_0) = \{a, b\}$. In general, we will use the standard notation for bags $\{\{a_1, \dots, a_n\}\}$ to denote the bag B whose identifiers are $I(B) = \{1, \dots, n\}$ and $B(i) = a_i$ for each $i \in I(B)$. We will use $\uplus$ to refer to the union of bags. Further, we define $\mathcal{P}_{\text{bags}}(A)$ as the set of all finite bags that one can form from a set A .

Computational model. We assume the model of *random access machines (RAM)* with uniform cost measure, and addition and subtraction as basic operations [1]. This implies, for example, that the access to a lookup table (i.e., a table indexed by a key) takes constant time. These are common assumptions in the literature of the area [3, 6].

2 The syntax and semantics of Complex Event Logic

In this section, we present the semantics of CEL [5].

Events and streams. We fix a countably infinite set of *attribute names* $\mathbf{A}$ and a countably infinite of *data values* $\mathbf{D}$ (e.g. integers, strings). An (untyped) *event* e is a pair (M, i) such that $M : \mathbf{A} \mapsto \mathbf{D}$ maps attribute names from $\mathbf{A}$ to data values in $\mathbf{D}$, and $i \in \mathbb{N}$ is the time of the event [4] (we prefer to use *discrete time*, which is enough for our purposes). Intuitively, M defines the data of the event (i.e., as a tuple). We denote by $e(\mathbf{a}) \in \mathbf{D}$ the value of the attribute $\mathbf{a} \in \mathbf{A}$ assigned by M (i.e., $e(\mathbf{a}) = M(\mathbf{a})$). If e is not defined on attribute $\mathbf{a}$, then we write $e(\mathbf{a}) = \text{NULL}$. Furthermore, for the sake of simplification we also denote $e(\text{time}) = i$ (note, however, that time is not an attribute). We define by $\text{Att}(e)$ the set of attributes of e , namely, $\text{Att}(e) = \text{dom}(M)$. We write $\mathbf{E}$ to denote the *set of all events* over attributes names

A and data values **D**. We will usually use bold letters **a**, **b**, and **c** to denote attribute names in **A** and (normal) letters *a*, *b*, and *c* to denote data values in **D**.

Fix now a finite set of *event types* **T** and assume that $\mathbf{T} \subseteq \mathbf{D}$ and $\text{NULL} \in \mathbf{D}$. In this work, we assume the existence of a distinguished attribute **type** that defines the *type* of an event. Specifically, let **type** be an attribute such that $\text{type} \in \mathbf{A}$. For every event *e*, we assume that $\text{type} \in \text{Att}(e)$ and $e(\text{type}) \in \mathbf{T} \cup \{\text{NULL}\}$ is the type of *e*. Notice that *e* could be *typed* (i.e., $e(\text{type}) \in \mathbf{T}$) or *untyped* in which case we have $e(\text{type}) = \text{NULL}$. A *schema* Σ is a function $\Sigma : \mathbf{T} \rightarrow \mathcal{P}(\mathbf{A}_\Sigma)$ where $\mathbf{A}_\Sigma \subseteq \mathbf{A}$ is a finite set of attributes. We say that an event *e* satisfies the schema Σ if, and only if, $\text{Att}(e) = \Sigma(e(\text{type})) \cup \{\text{type}\}$.

Let $\Sigma : \mathbf{T} \rightarrow \mathcal{P}(\mathbf{A}_\Sigma)$ be a schema. A *stream* over a schema Σ is an (arbitrary long) sequence $\mathcal{S} = e_1 e_2 \dots e_n$ of typed events such that, for every $i \in [n]$, it holds that $\text{Att}(e_i) = \Sigma(e_i(\text{type}))$ and $e_i(\text{time}) = i$. In other words, every event e_i is typed, satisfies the schema Σ , and its time is the position in the stream. Note that we defined the type and time of an event for its later use in the semantics of CEL. The first will allow us to know the attributes of each event when we compile CEL into an automata model, and the second will allow us to differentiate between tuples by adding its *origin* in the stream [2].

► **Example 1.** As a running example, consider that we have a stream $\mathcal{S}_{\text{Stocks}}$ that is emitting buy and sell events of particular stocks [3]. Here, we assume a schema Σ_{Stocks} with attributes **name** and **price** that represents the name of the stock (e.g., INTL for intel) and its price (e.g., US\$80), respectively. We have two types, called BUY and SELL, and $\Sigma_{\text{Stocks}}(\text{BUY}) = \Sigma_{\text{Stocks}}(\text{SELL}) = \{\text{name}, \text{price}\}$. A possible stream $\mathcal{S}_{\text{Stocks}}$ could be the following:

	0	1	2	3	4	5	6	7	8	9
$\mathcal{S}_{\text{Stocks}}$:	$\begin{bmatrix} \text{SELL} \\ \text{MSFT} \\ 101 \end{bmatrix}$	$\begin{bmatrix} \text{SELL} \\ \text{MSFT} \\ 102 \end{bmatrix}$	$\begin{bmatrix} \text{SELL} \\ \text{INTL} \\ 80 \end{bmatrix}$	$\begin{bmatrix} \text{BUY} \\ \text{INTL} \\ 80 \end{bmatrix}$	$\begin{bmatrix} \text{SELL} \\ \text{AMZN} \\ 1900 \end{bmatrix}$	$\begin{bmatrix} \text{SELL} \\ \text{INTL} \\ 81 \end{bmatrix}$	$\begin{bmatrix} \text{BUY} \\ \text{AMZN} \\ 1920 \end{bmatrix}$	$\begin{bmatrix} \text{BUY} \\ \text{MSFT} \\ 101 \end{bmatrix}$	$\begin{bmatrix} \text{BUY} \\ \text{INTL} \\ 79 \end{bmatrix}$	$\begin{bmatrix} \text{SELL} \\ \text{INTL} \\ 80 \end{bmatrix}$

Note that each event contains a type (i.e., BUY or SELL), its attributes values (i.e., **name** and **price**) and its time (i.e., the position above the event). Further, each event satisfies Σ_{Stocks} .

Predicates of events. A *predicate* is a possibly infinite set *P* of events. For instance, *P* could be the set of all events *e* such that $e(\mathbf{a}) \leq 20$. In our examples, we will use the notation $\mathbf{a} \sim a$ where $\mathbf{a} \in \mathbf{A}$, $a \in \mathbf{D}$, and $\sim$ is a binary relation over **D** to denote the predicate $P = \{e \mid e(\mathbf{a}) \sim a\}$. We say that an event *e* satisfies predicate *P*, denoted $e \models P$, if, and only if, $e \in P$. We generalize this notation from events to a bag of events *E* such that $E \models P$ if, and only if, $e \models P$ for every $e \in E$.

In this work, we assume a fix *set of predicates* **P** that is close under intersection, negation, and renaming, namely, $P_1 \cap P_2 \in \mathbf{P}$, $\mathbf{E} \setminus P \in \mathbf{P}$, and $r(P) \in \mathbf{P}$ for every $P, P_1, P_2 \in \mathbf{P}$ and $r \in \text{Ren}$ where $r(P) = \{r(e) \mid e \in P \wedge e \text{ is consistent with } r\}$ and **E**, the set of all events, is a predicate in **P** that we usually denote by **TRUE**.

Complex events. In this work, we will use a slightly different definition of complex event: we will store events inside valuations, instead of storing positions like in [3]. Formally, fix a finite set **X** of *variables*, which includes all event types (i.e. $\mathbf{T} \subseteq \mathbf{X}$). Let $\mathcal{S}$ be a stream of length *n*. A *complex event* of $\mathcal{S}$ is a triple (i, j, μ) where $i, j \in [n]$, $i \leq j$, and $\mu : \mathbf{X} \rightarrow \mathcal{P}_{\text{bags}}(\mathbf{E})$ is a function from variables to finite bags of events. Intuitively, *i* and *j* marks the beginning and end of the interval where the complex event happens, and μ stores the events in the interval $[i..j]$ that fired the complex event. In the following, we will usually denote *C* to denote a complex event (i, j, μ) of $\mathcal{S}$ and omit $\mathcal{S}$ if the stream is clear from the context. We will use $\text{time}(C)$, $\text{start}(C)$, and $\text{end}(C)$ to denote the interval $[i..j]$, the start *i*,

$$\begin{aligned}
\llbracket R \rrbracket(\mathcal{S}) &= \{(i, i, \mu) \mid i \in [n] \wedge e_i(\text{type}) = R \wedge \mu(R) = \{\{e_i\}\} \wedge \forall Y \neq R. \mu(Y) = \emptyset\} \\
\llbracket \varphi \text{ AS } X \rrbracket(\mathcal{S}) &= \{C \mid \exists C' \in \llbracket \varphi \rrbracket(\mathcal{S}). \text{time}(C) = \text{time}(C') \wedge C(X) = \biguplus_Y C'(Y) \\
&\quad \wedge \forall Z \neq X. C(Z) = C'(Z)\} \\
\llbracket \varphi \text{ FILTER } X[P] \rrbracket(\mathcal{S}) &= \{C \mid C \in \llbracket \varphi \rrbracket(\mathcal{S}) \wedge C(X) \models P\} \\
\llbracket \pi_L(\varphi) \rrbracket(\mathcal{S}) &= \{\pi_L(C) \mid C \in \llbracket \varphi \rrbracket(\mathcal{S})\} \\
\llbracket \varphi_1 \text{ OR } \varphi_2 \rrbracket(\mathcal{S}) &= \llbracket \varphi_1 \rrbracket(\mathcal{S}) \cup \llbracket \varphi_2 \rrbracket(\mathcal{S}) \\
\llbracket \varphi_1 \text{ AND } \varphi_2 \rrbracket(\mathcal{S}) &= \llbracket \varphi_1 \rrbracket(\mathcal{S}) \cap \llbracket \varphi_2 \rrbracket(\mathcal{S}) \\
\llbracket \varphi_1 : \varphi_2 \rrbracket(\mathcal{S}) &= \{C_1 \uplus C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\mathcal{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\mathcal{S}) \wedge \text{end}(C_1) + 1 = \text{start}(C_2)\} \\
\llbracket \varphi_1 ; \varphi_2 \rrbracket(\mathcal{S}) &= \{C_1 \uplus C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\mathcal{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\mathcal{S}) \wedge \text{end}(C_1) < \text{start}(C_2)\} \\
\llbracket \varphi \oplus \rrbracket(\mathcal{S}) &= \llbracket \varphi \rrbracket(\mathcal{S}) \cup \llbracket \varphi : \varphi \oplus \rrbracket(\mathcal{S}) \\
\llbracket \varphi + \rrbracket(\mathcal{S}) &= \llbracket \varphi \rrbracket(\mathcal{S}) \cup \llbracket \varphi ; \varphi + \rrbracket(\mathcal{S})
\end{aligned}$$

■ **Figure 1** The semantics of CEL defined over a stream $\mathcal{S} = e_1 e_2 \dots e_n$ where each e_i is an event.

and the end j of C , respectively. Further, by some abuse of notation we will also use $C(X)$ for $X \in \mathbf{X}$ to denote the bag $\mu(X)$ of C .

The following operations on complex events will be useful throughout the paper. We define the *union* of complex events C_1 and C_2 , denoted by $C_1 \uplus C_2$, as the complex event C' such that $\text{start}(C') = \min\{\text{start}(C_1), \text{start}(C_2)\}$, $\text{end}(C') = \max\{\text{end}(C_1), \text{end}(C_2)\}$, and $C'(X) = C_1(X) \uplus C_2(X)$ for every $X \in \mathbf{X}$. Further, we define the *projection over* $L \subseteq \mathbf{X}$ of a complex event C , denoted by $\pi_L(C)$, as the complex event C' such that $\text{time}(C') = \text{time}(C)$ and $C'(X) = C(X)$ whenever $X \in L$, and $C'(X) = \emptyset$, otherwise. Finally, we denote by $(i, j, \mu_\emptyset)$ the complex event with the trivial function $\mu_\emptyset$ such that $\mu_\emptyset(X) = \emptyset$ for every $X \in \mathbf{X}$.

Syntax and semantics for CEL. In this work, we use the *Complex Event Logic* (CEL) introduced in [5] and implemented in CORE [3] as our basic query language for CER. The syntax of a CEL formula is given by the following grammar:

φ	$:=$	R	(event type selection)		$\varphi \text{ AS } X$	(variable binding)
		$\varphi \text{ FILTER } X[P]$	(predicate filtering)		$\pi_L(\varphi)$	(variable projection)
		$\varphi \text{ OR } \varphi$	(disjunction)		$\varphi \text{ AND } \varphi$	(conjunction)
		$\varphi : \varphi$	(contiguous sequencing)		$\varphi ; \varphi$	(non-cont. sequencing)
		$\varphi \oplus$	(contiguous iteration)		$\varphi +$	(non-cont. iteration)

where R is an event type, $X \in \mathbf{X}$ is a variable, $P \in \mathbf{P}$ is a predicate, and $L \subseteq \mathbf{X}$ is a finite set of variables. Similar to [5, 3], we define the semantics of a CEL formula φ over a stream $\mathcal{S} = e_1 e_2 \dots e_n$, recursively, as a set of complex events over $\mathcal{S}$. The main difference is the notion of complex events, that now contains events instead of positions. In Figure 1, we define the semantics of each CEL operator like in [3, 5]. Given a formula φ , the semantics $\llbracket \varphi \rrbracket(\mathcal{S})$ defines a set of complex events. Notice that $\llbracket \varphi \rrbracket(\mathcal{S})$ has a *set-semantics* and, instead, complex events store bags of events.

3 New operators in CEL

For defining the new operators, we consider a subfragment of CEL formulas, called *atomic CEL formulas*, given by the following grammar:

α	$:=$	R	(event type selection)		α AS X	(variable binding)
		α FILTER $X[P]$	(predicate filtering)		$\pi_L(\alpha)$	(variable projection)
		α OR α	(disjunction)		α AND α	(conjunction)

Note that for every atomic formula α , $\llbracket \alpha \rrbracket(\mathcal{S})$ captures complex events with a single event, namely, $C = (i, i, \mu)$ for every $C \in \llbracket \alpha \rrbracket(\mathcal{S})$ and some $i \in \mathbb{N}$ and mapping μ .

First, we define the *not*-operator $!\alpha$ of an atomic formula α over a stream $\mathcal{S} = e_1 e_2 \dots e_n$ as follows:

$$\llbracket !\alpha \rrbracket(\mathcal{S}) = \{(i, i, \mu_\emptyset) \mid i \in [k] \wedge \forall C \in \llbracket \alpha \rrbracket(\mathcal{S}). \text{time}(C) \neq [i, i]\}$$

Namely, it captures all complex events of a single event e_i that are not captured by α .

By using the not-operator, we can define the **NEXT** and **LAST** operator of an atomic formula α as follow:

$$\begin{aligned} \text{NEXT}(\alpha) &\equiv (!\alpha) \oplus : \alpha \text{ OR } \alpha \\ \text{LAST}(\alpha) &\equiv (\alpha : (!\alpha) \oplus) \text{ OR } \alpha \end{aligned}$$

Intuitively, the **NEXT** operator finds the first time that α holds, and **LAST** finds the last (backwardsly) time that α holds.

References

- 1 Alfred V Aho and John E Hopcroft. *The design and analysis of computer algorithms*. Pearson Education India, 1974.
- 2 Mikołaj Bojańczyk. Transducers with origin information. In *Automata, Languages, and Programming: 41st International Colloquium, ICALP 2014, Copenhagen, Denmark, July 8-11, 2014, Proceedings, Part II 41*, pages 26–37. Springer, 2014.
- 3 Marco Bucci, Alejandro Grez, Andrés Quintana, Cristian Riveros, and Stijn Vansummeren. CORE: a complex event recognition engine. *VLDB*, 15(9):1951–1964, 2022.
- 4 Julián García and Cristian Riveros. Complex event recognition under time constraints: Towards a formal framework for efficient query evaluation. *Proc. ACM Manag. Data*, 3(2):94:1–94:17, 2025.
- 5 Alejandro Grez, Cristian Riveros, Martín Ugarte, and Stijn Vansummeren. A formal framework for complex event recognition. *ACM TODS*, 46(4):16:1–16:49, 2021.
- 6 Luc Segoufin. Enumerating with constant delay the answers to a query. In *Proceedings of the 16th International Conference on Database Theory*, pages 10–20, 2013.